

AEMB: A Deterministic Cross-Language Verification Benchmark for Event-Margin Protocols

Maksym Nechepurenko*

August 2026

Abstract

Event-margin protocols span financial accounting, venue evidence, smart contracts, services, user interfaces, and operational recovery. Unit tests within one implementation cannot establish that a Python model, Solidity contracts, TypeScript services, and venue adapters preserve the same meaning under the same event sequence. This paper introduces the Axiom Event-Margin Protocol Verification Benchmark (AEMB), a deterministic and implementation-neutral conformance benchmark for that cross-layer problem. AEMB v0.1.0 contains 38 source-generated scenarios anchored to an immutable reference implementation: eight financial vertical slices, fifteen venue-adaptor boundary cases, and fifteen operational or fault rehearsals. Each case declares preconditions, typed actions, expected transitions, balance conventions, outcome hashes, negative controls, and invariant mappings. Conformance is conjunctive: a mandatory schema, transition, hash, safety-boundary, or reproducibility failure cannot be offset by success elsewhere. Python, Node.js, and Go canonicalizers agree on 11,400 recursive key-order comparisons with zero failures; a strict offline Draft 2020-12 schema accepts all canonical cases and rejects twelve malformed controls; and 190 one-field complete-case mutations all change case identity. The benchmark separates outcome, case, collection, dataset, and package identities and defines correction, extension, licensing, and archival rules. AEMB is synthetic and deterministic. It contains no raw third-party venue data, user identifiers, external writes, or production asset movement, and it does not certify production safety or live venue compatibility. Its contribution is a portable benchmark object, source-anchored public release, exact cross-language evidence, and a governance protocol for future cases derived from audits, incidents, and empirical traces.

*Founder and Director of Research, ForesightFlow, the Research Department of Devnull FZCO, Dubai, United Arab Emirates. Email: maksym@devnull.ae.

Keywords: deterministic benchmark; cross-language verification; smart contracts; financial protocols; canonical JSON; mutation testing; event sourcing.

ACM CCS: Software and its engineering—Software testing and debugging; Security and privacy—Software security engineering.

JEL Classification: C63, G13, G14, G23.

1 Motivation

Financial protocols are implemented as systems, not single programs. A position may be admitted by a Python risk model, opened by Solidity contracts, orchestrated by a TypeScript service, reflected in a browser projection, and reconstructed by an audit tool. Passing tests within each layer does not prove semantic agreement across layers. The same field name can carry different units, the same event can be applied twice, a match can be treated as cash, or a browser can display a state that the contract never reached.

Differential testing, property-based testing, mutation analysis, metamorphic testing, and executable semantics address parts of this problem [McKeeman, 1998, Claessen and Hughes, 2000, Jia and Harman, 2011, Chen et al., 1998, Hildenbrandt et al., 2018]. General benchmark suites such as MLPerf and BIG-bench demonstrate the value of immutable task definitions and explicit evaluation protocols [Reddi et al., 2020, BIG-Bench Collaboration, 2022]. AEMB adapts those ideas to financially meaningful event-margin transitions.

The benchmark is intentionally implementation-neutral. It does not require Axiom source code, an on-chain deployment, or a specific venue. It specifies a canonical scenario object and observable obligations. A conforming implementation must map its own commands, storage, and events to that object without changing its meaning.

The concise revision reports the public v0.1.0 benchmark, its formal identity model, exact cross-language evidence, and principal coverage limitations. Extended registries and release checklists remain available in the earlier arXiv version history and public dataset package.

2 Benchmark object

Definition 2.1 (AEMB case). *An AEMB case is a tuple*

$$C = (I, P, A, T, O, N, M), \tag{1}$$

where I is immutable identity metadata, P preconditions, A an ordered typed action sequence, T required transitions, O the expected terminal oracle, N negative controls and forbidden effects, and M implementation and invariant mappings.

AEMB publication and conformance architecture

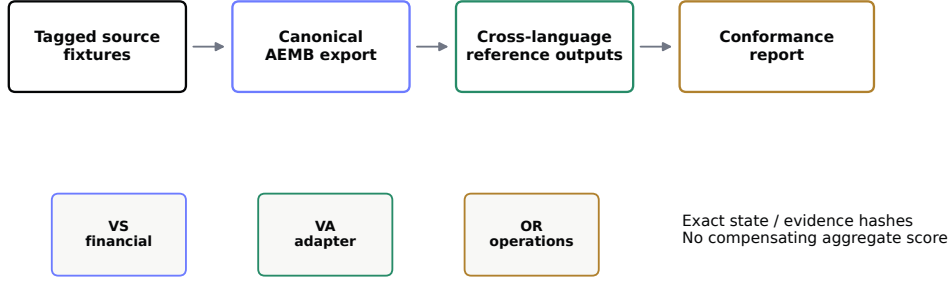


Figure 1: AEMB separates a canonical scenario object from implementation-specific adapters and execution layers.

The terminal oracle contains complete financially relevant state, not only a pass flag. It may include collateral, debt principal and interest, matched and settled capacity, provider shares, reserve and LBP balances, withdrawal queue state, evidence status, pause state, and expected event identifiers.

Canonicalization follows RFC 8785-style JSON canonicalization [Rundgren et al., 2020]. Let $\text{Canon}(x)$ be UTF-8 canonical JSON and SHA256 SHA-256.

Definition 2.2 (Identity layers). *For case C :*

$$h_O = \text{SHA256}(\text{Canon}(O)), \quad \text{registered outcome identity}, \quad (2)$$

$$h_C = \text{SHA256}(\text{Canon}(C)), \quad \text{complete-case digest}. \quad (3)$$

A collection identity hashes the ordered vector of case digests. A dataset identity hashes the versioned collection manifest. A package identity hashes the distributable archive and may change without changing semantic data.

Proposition 2.3 (Path information is not implied by outcome identity). *Two cases can have the same terminal oracle O but different action sequences, evidence paths, or forbidden transitions. Hence equal h_O does not imply equal h_C .*

Proposition 2.4 (Case mutation sensitivity). *If one semantically included field of C changes while canonicalization and all other fields remain fixed, collision resistance implies that the complete-case digest changes except with negligible probability.*

3 Collections and scenario coverage

AEMB v0.1.0 contains 38 scenarios in three collections.

Table 1: AEMB v0.1.0 composition.

Collection	Scope	Cases
VS	Financial vertical slices	8
VA	Venue-adaptor boundaries	15
OR	Operational and fault rehearsals	15
Total		38

Financial vertical slices (VS). These cases cover admitted and rejected leveraged positions, partial and final settlement, debt-first residual allocation, liquidation and waterfall ordering, Senior withdrawal queues, LBP impairment, and exactly-once recovery. They are designed to expose accounting differences across model, contract, service, and UI implementations.

Venue boundaries (VA). These cases cover snapshot and incremental book reconstruction, sequence gaps, stale data, capability expiry, partial fills, match versus settlement, settlement failure, actual close, resolution, redemption, signer restrictions, and external-write prohibitions. A recorded fixture demonstrates an interface boundary, not venue approval.

Operational and fault rehearsals (OR). These cases cover duplicate and conflicting evidence, worker restart, database restore, projection rebuild, RPC or local-chain outage, pause and recovery, withdrawal surges, telemetry loss, credential boundaries, and orderly wind-down.

The public v0.1.0 release intentionally leaves `claim_ids` empty. It supports invariant and scenario conformance, not claim-level traceability to a particular paper or product assertion.

3.1 Compact scenario registry

The release IDs are stable and grouped by purpose.

4 Conformance protocol

A submission declares an implementation identity, compiler/runtime versions, mapping from AEMB actions to local entrypoints, and mapping from local outputs to the canonical terminal oracle. Evaluation proceeds in layers.

Table 2: Scenario families in AEMB v0.1.0.

IDs	Boundary	Representative obligations
VS-001–VS-003	Origination and admission	$1 \times / 2 \times$ admission, rejection, collateral and debt creation.
VS-004–VS-006	Settlement and liquidation	Partial settlement, debt-first close, liquidation, reserve/LBP/Senior ordering.
VS-007–VS-008	Provider and recovery	Loss-participating queue, evidence replay/conflict, exactly-once recovery.
VA-001–VA-005	Market data	Snapshot, delta, sequence gap, stale data, resnapshot.
VA-006–VA-010	Order and settlement	Submit/cancel, partial fill, match, settlement pending/failure.
VA-011–VA-015	Lifecycle/capability	Close, resolution, redemption, signer/capability restrictions.
OR-001–OR-005	Evidence and workers	Duplicate/conflict, crash/restart, durable inbox/outbox.
OR-006–OR-010	Infrastructure	Database restore, RPC outage, projection rebuild, telemetry failure.
OR-011–OR-015	Governance and capital	Withdrawal surge, reserve/LBP stress, pause, recovery, wind-down.

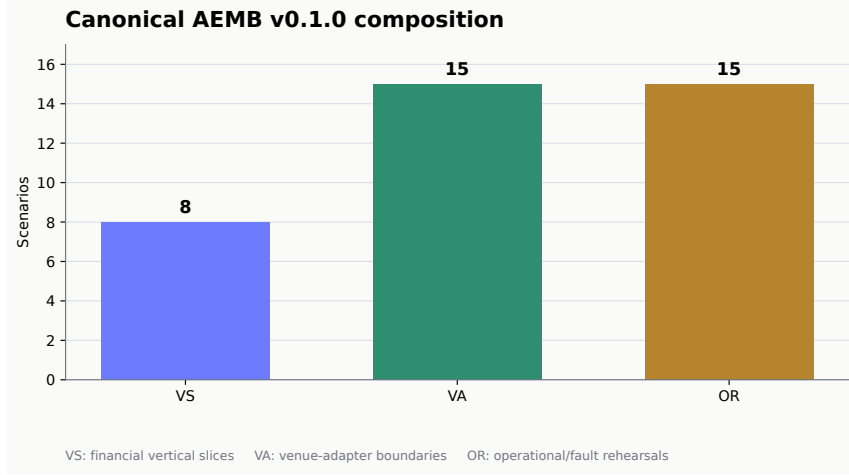


Figure 2: Balanced collection design across financial, venue, and operational boundaries.

4.1 Schema and canonicalization

Every case must validate against the offline Draft 2020-12 schema [Wright et al., 2022]. Implementations must produce the same canonical bytes for recursive object-key permutations. Array order is semantic and must not be reordered.

4.2 Transition conformance

The implementation executes the ordered action sequence from a clean precondition state. Required transitions must occur; forbidden financial effects must not occur. Negative cases can pass by rejecting a command or quarantining evidence, but the rejection and zero financial mutation must be observable.

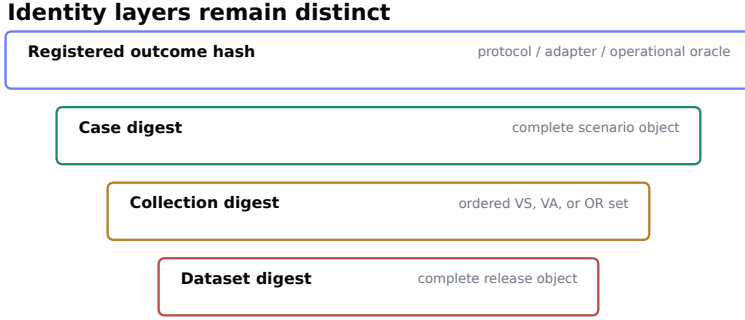


Figure 3: Outcome, complete-case, collection, dataset, and package identities answer different reproducibility questions.

4.3 Terminal and path conformance

The mapped terminal oracle must equal the registered oracle in units, rounding, and missing-value semantics. When a case registers event or receipt identities, the execution path must also satisfy them. A correct final balance obtained through an unauthorized intermediate transfer is non-conforming.

4.4 Reproducibility

At least two clean runs must produce identical normalized case results. Runtime-only fields such as wall-clock timestamps, process identifiers, and transaction hashes may be excluded only if the normalization policy is declared before execution and the semantic links remain verifiable.

Definition 4.1 (Conjunctive conformance). *A case conforms if and only if all mandatory schema, transition, terminal, path, boundary, and reproducibility predicates are true. Dataset conformance is reported per case and per collection; no aggregate score can compensate for a mandatory safety failure.*

5 Cross-language and negative-control evidence

The public release includes canonicalizers in Python, Node.js, and Go. For every case, 100 recursive key-order permutations are generated independently. All three implementations produce the same complete-case digest:

$$38 \times 100 \times 3 = 11,400 \quad (4)$$

comparisons with zero failures.

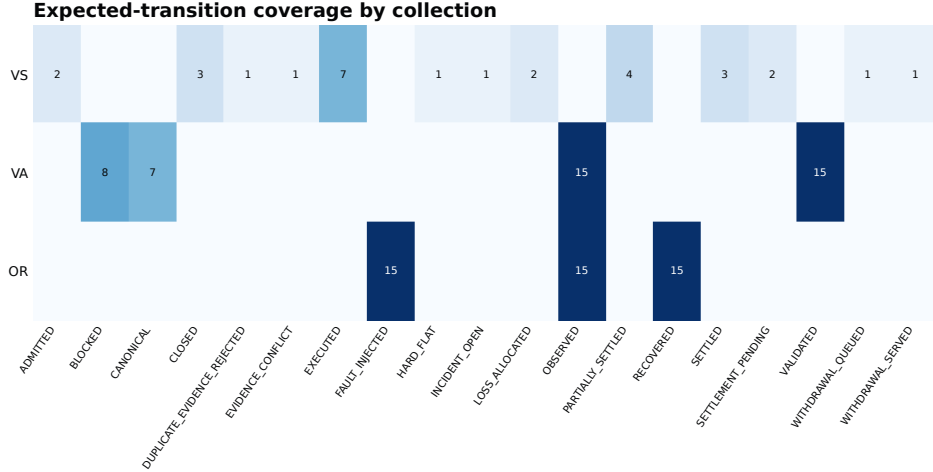


Figure 4: Registered transition coverage across scenario families. Darker cells indicate repeated exercise of the same transition class.

Table 3: Registered validation evidence.

Check	Result
Canonical cases accepted	38 / 38
Malformed controls rejected	12 / 12
One-field identity mutations detected	190 / 190
Python/Node.js/Go key-order comparisons	11,400 / 11,400
Manifest entries verified	all
External writes / production asset movement	0 / 0

The strict schema accepts all 38 canonical cases and rejects twelve malformed controls. Controls include missing identity fields, wrong types, unknown action variants, malformed hashes, invalid leverage boundaries, and structurally incomplete expected state.

Five one-field mutations are applied to every complete case, for 190 mutations in total. Mutated fields span action parameters, expected state, negative controls, provenance, and implementation mapping. All 190 change complete-case identity.

6 Reference algorithms

The public release contains simple reference procedures whose purpose is transparency rather than performance.

6.1 Canonical case identity

Given a parsed case object C , the canonicalizer:

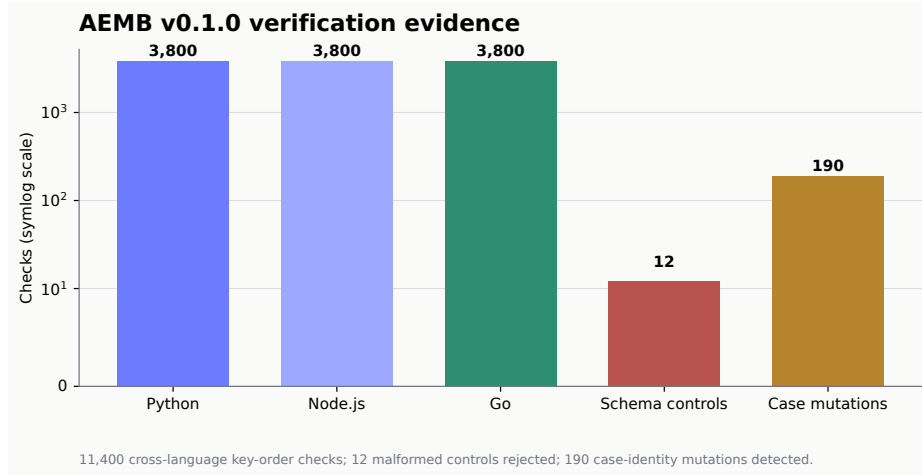


Figure 5: Exact cross-language agreement under recursive key-order permutations.

- (1) verifies the declared schema version and rejects unknown required variants;
- (2) normalizes integers and fixed-point quantities to their registered string representation;
- (3) preserves array order and recursively sorts object keys;
- (4) emits UTF-8 canonical JSON without insignificant whitespace; and
- (5) computes SHA-256 over the resulting bytes.

The implementation must not round, coerce, or default a field during hashing. Validation and canonicalization are separate: an invalid object can have a hash but is not an AEMB case.

6.2 Scenario runner

A generic runner proceeds as follows:

- (1) create a clean implementation state and verify preconditions;
- (2) map each AEMB action to a declared local entrypoint;
- (3) record command, transaction, event, and journal identities;
- (4) apply actions in order, stopping only when the case specifies fail-closed rejection;
- (5) reconstruct the local terminal state through the submission's mapping;
- (6) compare the mapped state, required transitions, and forbidden effects with the registered case;
- (7) produce a normalized per-case result containing every predicate and mismatch.

A runner may use a simulator, local chain, service stack, or venue emulator, but must state which boundaries are actually executed. Loading a stored fixture is not equivalent to executing the corresponding contract or recovery path.

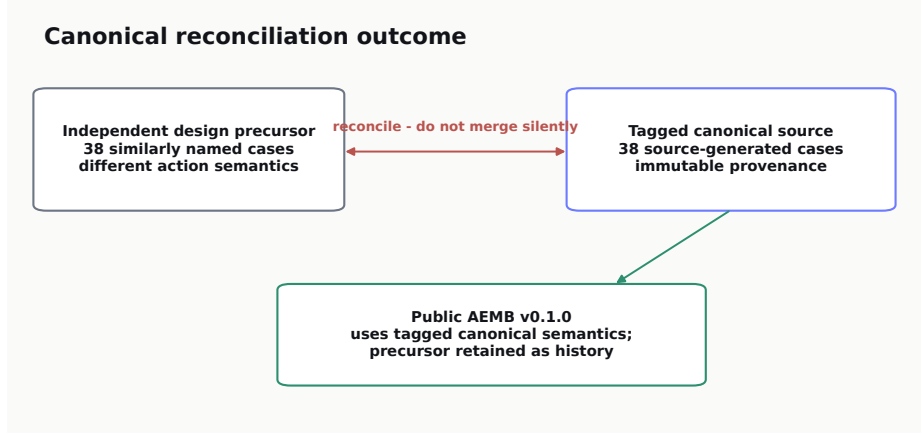


Figure 6: Reconciliation separates exact match, semantic match with different identity, additive cases, conflicts, and rights-blocked material.

6.3 Clean-run normalization

Raw runs can differ in wall-clock time, process identifiers, temporary paths, and transaction hashes. A normalization policy may remove such fields only when they are non-semantic and alternative correlation evidence remains. Financial quantities, transition order, event topics, evidence identities, state hashes, and pass/fail predicates are always semantic. The normalized result digest is

$$h_R = \text{SHA256}(\text{Canon}((R_1, \dots, R_{38}))). \quad (5)$$

Two clean runs conform to the reproducibility gate when their normalized objects are byte-identical and all registered predicates pass.

6.4 Mutation and metamorphic checks

Structural malformed controls test rejection. One-field identity mutations test that complete-case digest includes the intended field. Domain semantic mutants test whether the corpus distinguishes financially different behavior, such as interest/principal priority or reserve/LBP ordering. Metamorphic tests vary object-key order, harmless serialization choices, and runtime while requiring the same identity.

These layers answer different questions. Schema controls show that malformed records are not accepted; identity mutations show that recorded fields are committed; semantic mutants probe coverage; and cross-language permutations show canonicalization consistency. None alone proves implementation correctness.

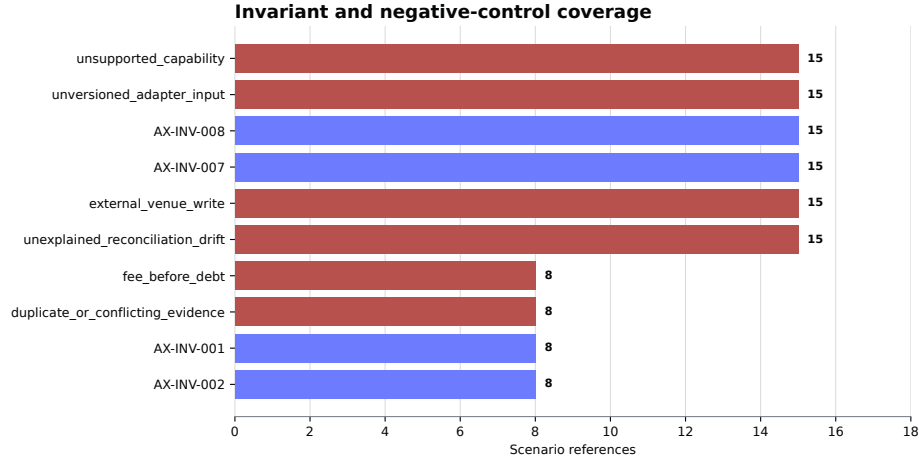


Figure 7: Coverage is reported by invariant and negative boundary, not collapsed into one compensating score.

7 Coverage analysis and mutation debt

Benchmark completeness is not inferred from the number of passing implementations. AEMB uses action and invariant coverage, negative controls, and domain-specific semantic mutants to identify missing distinctions.

The initial mutation study included eighteen semantic mutants. Twelve were killed by existing cases. Six survived, exposing concrete coverage debt:

- (1) settlement exceeding matched capacity;
- (2) interest-before-principal under insufficient proceeds;
- (3) incorrect LBP/global-reserve order under partial absorption;
- (4) origination without sufficient collateral;
- (5) incident handling while debt remains positive; and
- (6) attempted external venue write.

These survivors motivate additive scenarios rather than retrospective reinterpretation of v0.1.0.

AEMB also avoids overclaiming from empty fields. The current release does not populate claim-level identifiers, does not measure gas or performance, and does not contain empirical venue observations. Those dimensions may appear in later minor versions with explicit schema changes.

8 Integration and governance

AEMB can be used in several ways:

- differential testing across Python, Solidity, and service implementations;

Release governance is part of benchmark correctness



Any semantic change creates a new dataset identity; published files are never overwritten silently.

Figure 8: Source-anchored release pipeline from scenario export through validation, identity freeze, and archival publication.

- audit-scope regression after remediation;
- venue-adaptor conformance without external writes;
- release gating for accounting and evidence changes;
- public comparison of independent implementations; and
- minimization of long AVET traces into compact protocol cases.

Versioning follows semantic rules. A major version changes canonical semantics or compatibility. A minor version adds backward-compatible cases or fields. A patch changes metadata or packaging without changing case meaning. Published files are not silently replaced. Corrections receive a new version, manifest, and explanation.

The public dataset and expected outputs are licensed CC BY 4.0; schemas, canonicalizers, validators, and reference code are Apache-2.0. Axiom marks are separately governed. Raw third-party data are not part of the release.

The release archive has SHA-256

bf44e291ad47337dad6ec4ddb64d5fcec4435e07261fe9ec86321329d16cda7b,

and semantic dataset digest

8a23fd013a2d484b666da1a2a3b607234f3fc5b33a887b636136798984220467.

The distinction matters: metadata packaging can change while semantic case identity remains fixed.

9 Submission profiles and comparative use

A benchmark submission should identify the implementation under test rather than upload an opaque score. The minimum profile contains:

- repository or source identity and license;

- language, compiler, runtime, and dependency-lock hashes;
- mapping from AEMB actions to commands and from local state to the canonical oracle;
- scenario-level pass/fail results and raw mismatch reports;
- normalization policy for runtime-only fields;
- two clean-run identities; and
- a declaration of excluded collections or unsupported actions.

Partial conformance is allowed but must be explicit. For example, a venue adapter can report VA conformance without claiming VS accounting conformance; a contract package can report VS without pretending to validate browser or operational behavior. The comparison unit is the scenario and invariant, not a single leaderboard number.

Future public submissions may be verified by a reference runner or independently reproduced by a third party. Any leaderboard should preserve failed safety cases, implementation version, hardware/runtime context, and evidence links. Performance metrics such as gas, latency, or throughput are separate dimensions and cannot compensate for semantic failure.

From incidents to benchmark cases. A new case derived from an audit finding or incident should be minimized while preserving the violated invariant. The release record should state the source class, whether details were sanitized, the regression expectation, and whether the case is public immediately or embargoed for responsible disclosure. If derived from an AVET trace, the trace digest and minimization mapping should be retained without copying restricted raw data.

10 Threats to validity

AEMB can fail in several ways even when every registered case passes. First, the benchmark and an implementation may share the same misunderstanding of a financial rule. Cross-language agreement detects serialization and implementation divergence, not a common specification error. Independent audits and proofs therefore remain complementary.

Second, case selection is endogenous to the current protocol design. The benchmark emphasizes debt, settlement evidence, provider claims, queues, venue capabilities, and recovery because those are the identified high-risk surfaces. Other protocols may require additional objects such as portfolio margin, cross-collateral netting, multi-outcome settlement, or governance voting. Unsupported scope should be declared rather than forced into an ill-fitting case.

Third, a complete terminal oracle can still miss a harmful transient state. Path

predicates and required events reduce this risk, but only the transitions explicitly registered are observable. Future releases should add invariants discovered through formal verification, adversarial review, and incident minimization.

Fourth, public benchmarks invite overfitting. An implementation can special-case known scenarios while remaining incorrect elsewhere. Property and mutation tests, hidden extension cases, and independently derived AVET traces can mitigate but not eliminate this problem. AEMB should therefore be used as a release floor and comparison protocol, not as the sole security argument.

Finally, reproducibility depends on the declared normalization boundary. Removing transaction hashes, timestamps, or runtime identifiers can be legitimate, but an overly broad normalizer can erase a semantic difference. Every excluded field must have a reason, and alternative evidence must preserve the cross-layer correlation it carried.

11 Limitations

AEMB is deterministic and synthetic or sanitised. It does not measure production safety, live venue compatibility, market volume, user behavior, latency, yield, default frequency, reserve sufficiency, or regulatory compliance. Passing AEMB means that an implementation preserved the registered semantics for the registered cases; unregistered failure modes remain possible.

The benchmark is also finite. Its action language and terminal oracle reflect the current event-margin design. A malicious or simply different implementation may exploit an unobserved intermediate state. Mutation analysis reduces this risk but cannot prove completeness. Cross-language agreement can also reproduce a shared specification error.

Finally, benchmark governance can create incentives to overfit. New cases should therefore be derived from independently identified gaps, audit findings, or rights-cleared traces, with versioned holdout and disclosure policies where comparative evaluation is intended.

12 Conclusion

AEMB turns a cross-layer financial protocol into a portable set of deterministic obligations. It separates path and terminal identity, rejects compensating aggregate scores, and requires exact semantics across languages and runtime boundaries. The public v0.1.0 release shows that 38 source-generated cases can be canonicalized consistently in Python, Node.js, and Go, validated offline, challenged by malformed inputs, and protected by complete-case mutation identities.

The benchmark is not a substitute for formal verification, audits, or live evidence. Its role is narrower and practical: make disagreements reproducible, make release claims inspectable, and preserve the semantics of critical accounting and failure paths across implementations. Future versions can add coverage without rewriting the identity of the initial public corpus.

Data and code availability

AEMB v0.1.0 is available at <https://github.com/AxientLab/aemb/releases/tag/aemb-v0.1.0>. DOI information will be added in a later revision.

Disclosure

Automated language and code-assistance tools were used for editorial review, schema and validator development, consistency checking, and figure production. The author specified and reviewed the benchmark object, cases, identities, evaluation protocol, interpretation, and final manuscript, and takes responsibility for the work.

References

- BIG-Bench Collaboration. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models, 2022.
- Tsong Yueh Chen, Shing Chi Cheung, and Shiu Ming Yiu. Metamorphic testing: A new approach for generating next test cases. *Technical Report HKUST-CS98-01*, 1998.
- Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279, 2000. doi: 10.1145/357766.351266.
- Everett Hildenbrandt, Manasvi Saxena, Nishant Rodrigues, Xiaoran Zhu, Philip Daian, Dwight Guth, and Grigore Rosu. KEVM: A complete formal semantics of the ethereum virtual machine. In *31st IEEE Computer Security Foundations Symposium*, pages 204–217, 2018. doi: 10.1109/CSF.2018.00022.
- Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10 (1):100–107, 1998.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf inference benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, pages 446–459, 2020. doi: 10.1109/ISCA45697.2020.00045.

Anders Rundgren, Bret Jordan, and Samuel Erdtman. RFC 8785: Json canonicalization scheme (JCS). RFC Editor, 2020.

Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. Json schema draft 2020-12. JSON Schema specification, 2022.